Pastel Atributos de Calidad según SQUARE - 25010

🔹 1. Funcionalidad (Functionality)

Capacidad del software para cumplir con los requisitos funcionales especificados.

📌 **Subatributos:**

- **Integridad funcional**: Asegura que todas las funciones trabajen de forma integrada y sin inconsistencias.
  - *Ejemplo:* Un **ERP empresarial** permite la sincronización de inventarios y ventas sin pérdida de datos.
- **Corrección funcional**: Capacidad del sistema para producir resultados correctos según lo especificado.

    - *Ejemplo:* Un **sistema contable** debe calcular correctamente impuestos y descuentos sin errores.
- **Adecuación funcional**: Cumple con las necesidades del usuario sin necesidad de personalización adicional.
    - *Ejemplo:* Un **CRM** que permite gestionar clientes sin necesidad de modificar la funcionalidad base.

🔹 2. Rendimiento (Performance Efficiency)

Capacidad del software para ejecutar sus funciones de manera eficiente.

📌 **Subatributos:**

- **Comportamiento temporal**: Rapidez con la que se ejecutan las operaciones.
    - *Ejemplo:* **Google Search** responde a una consulta en menos de 0.5 segundos.
- **Utilización de recursos**: Optimización del uso de CPU, memoria, red y almacenamiento.
    - *Ejemplo:* **Netflix** ajusta la calidad del video según el ancho de banda disponible.
- **Capacidad**: Habilidad del sistema para manejar cargas de trabajo sin degradación.
    - *Ejemplo:* **Amazon AWS** escala servidores automáticamente en eventos de alto tráfico.

🔹 3. Compatibilidad (Compatibility)

Capacidad del software para operar en diferentes entornos y con otros sistemas.

📌 **Subatributos:**

- **Coexistencia**: Puede ejecutarse junto con otros sistemas sin interferencias.
    - *Ejemplo:* **Microsoft Office y Google Docs** pueden instalarse en la misma computadora sin conflictos.
- **Interoperabilidad**: Comunicación fluida con otros sistemas o plataformas.
    - *Ejemplo:* **Zapier** permite conectar Slack, Gmail y Trello para automatizar flujos de trabajo.

🔹 4. Usabilidad (Usability)

Facilidad de uso y accesibilidad del software.

📌 **Subatributos:**

- **Reconocimiento de adecuación**: Los usuarios pueden identificar fácilmente que la herramienta cumple sus necesidades.
  - *Ejemplo:* **Figma** muestra plantillas para ayudar a diseñadores a comenzar rápidamente.
- **Capacidad de aprendizaje**: Facilidad con la que nuevos usuarios pueden aprender a usar el software.
  - *Ejemplo:* **Notion** tiene tutoriales interactivos que facilitan la adopción.
- **Operatividad**: Facilidad de uso sin necesidad de conocimientos avanzados.
  - *Ejemplo:* **Google Docs** tiene una interfaz simple que cualquier usuario puede manejar intuitivamente.
- **Protección de errores**: Evita que los usuarios cometan errores o los corrige automáticamente.
  - *Ejemplo:* **Excel** sugiere correcciones si detecta una fórmula mal escrita.
- **Estética de la interfaz**: Diseño visual atractivo y funcional.
  - *Ejemplo:* **Apple Music** utiliza una interfaz limpia y visualmente atractiva.
- **Accesibilidad**: Diseño que permite el uso por personas con discapacidades.
  - *Ejemplo:* **YouTube** permite activar subtítulos automáticos y tiene soporte para lectores de pantalla.

🔹 5. Fiabilidad (Reliability)

Capacidad del software para operar sin fallos y recuperar su funcionamiento en caso de fallos.

📌 **Subatributos:**

- **Madurez**: Probabilidad de fallos en condiciones normales de operación.
  - *Ejemplo:* **Linux** es conocido por su estabilidad y bajos fallos en entornos de servidores.
- **Disponibilidad**: Porcentaje de tiempo que el sistema está operativo sin interrupciones.
  - *Ejemplo:* **Amazon AWS** tiene una disponibilidad del 99.99%.
- **Tolerancia a fallos**: Capacidad del sistema para seguir operando tras un fallo.
  - *Ejemplo:* **Google Drive** guarda automáticamente los cambios en documentos para evitar pérdidas de datos.
- **Capacidad de recuperación**: Rapidez con la que el sistema puede restablecerse tras un fallo.
  - *Ejemplo:* Un **servidor de streaming** puede recuperarse en segundos tras una caída usando balanceo de carga.

🔹 6. Seguridad (Security)

Capacidad del software para proteger la información contra accesos no autorizados y ataques.

📌 **Subatributos:**

- **Confidencialidad**: Protege los datos del acceso no autorizado.
  - *Ejemplo:* **WhatsApp** usa cifrado de extremo a extremo en mensajes.
- **Integridad**: Asegura que los datos no sean alterados sin permiso.
  - *Ejemplo:* Un **sistema bancario** usa firmas digitales para prevenir fraudes.
- **No repudio**: Evita que los usuarios nieguen haber realizado acciones.
  - *Ejemplo:* **Correos electrónicos en Gmail** tienen registros de envío con timestamps para auditoría.
- **Responsabilidad**: Registro de todas las acciones realizadas en el sistema.
  - *Ejemplo:* Un **ERP** registra qué usuario modificó cada dato.
- **Autenticidad**: Verifica la identidad de los usuarios.
  - *Ejemplo:* **Facebook** usa autenticación de dos factores (2FA) para accesos seguros.

🔹 7. Mantenibilidad (Maintainability)

Facilidad con la que el software puede modificarse, corregirse o mejorarse.

📌 **Subatributos:**

- **Modularidad**: División en componentes independientes para facilitar cambios.
  - *Ejemplo:* **Django** permite desarrollar aplicaciones en módulos reutilizables.
- **Reutilización**: Capacidad de usar componentes en diferentes proyectos.
  - *Ejemplo:* **Bootstrap** ofrece componentes reutilizables para interfaces web.
- **Analizabilidad**: Facilidad para diagnosticar errores o mejorar código.
  - *Ejemplo:* **SonarQube** analiza código en busca de errores y problemas de seguridad.
- **Modificabilidad**: Facilidad para realizar cambios sin afectar otras partes del sistema.
  - *Ejemplo:* Un **CMS** como WordPress permite modificar temas sin afectar la funcionalidad base.
- **Testeabilidad**: Facilidad para probar y verificar el software.
  - *Ejemplo:* **JUnit** permite ejecutar pruebas automatizadas en aplicaciones Java.

🔹 8. Portabilidad (Portability)

Capacidad del software para ejecutarse en diferentes entornos con mínimas modificaciones.

📌 **Subatributos:**

- **Adaptabilidad**: Capacidad para ajustarse a distintos dispositivos o sistemas operativos.
    - *Ejemplo:* **Spotify** funciona en Windows, macOS, Android y iOS con la misma experiencia de usuario.
- **Facilidad de instalación**: Simplicidad en la instalación y configuración del sistema.
    - *Ejemplo:* **Docker** permite instalar aplicaciones en contenedores con configuraciones predefinidas.
- **Intercambiabilidad**: Facilidad para reemplazar un sistema por otro sin afectar a los usuarios.
    - *Ejemplo:* **MySQL puede ser reemplazado por PostgreSQL** en muchas aplicaciones con mínimos cambios.

🎯 Conclusión

Los **atributos de calidad** son esenciales para diseñar software robusto y exitoso. Cada sistema debe priorizar aquellos que sean más relevantes según su propósito y usuarios.

🔥 Matriz de Tensiones entre Atributos de Calidad

La **Matriz de Tensiones** en arquitectura de software es una herramienta que muestra **las relaciones conflictivas entre los atributos de calidad**. Dado que **no se pueden optimizar todos los atributos al mismo tiempo**, esta matriz ayuda a visualizar **qué atributos entran en conflicto** cuando se toman decisiones arquitectónicas.

🔹 1. ¿Por qué existen tensiones entre atributos de calidad?

Cada atributo de calidad tiene **requisitos que pueden entrar en conflicto** con otros. Por ejemplo:

- **Seguridad vs Rendimiento**: Agregar autenticación multifactor (MFA) mejora la seguridad, pero **ralentiza el acceso** al sistema.

- **Disponibilidad vs Mantenibilidad**: Un sistema con alta disponibilidad (99.99%) **dificulta las actualizaciones** porque nunca puede apagarse para mantenimiento.
- **Escalabilidad vs Consistencia**: Un sistema distribuido puede escalar mejor, pero mantener **consistencia fuerte** (ACID) en bases de datos distribuidas es difícil.

🔹 2. Matriz de Tensiones: Conflictos entre Atributos de Calidad

Esta matriz muestra los conflictos más comunes entre los principales atributos:

Atributos	Fiabilidad	Seguridad	Rendimiento	Escalabilidad	Mantenibilidad	Usabilidad
Fiabilidad	✅ Compatible	⚠️ Agregar validaciones puede reducir disponibilidad	⚠️ Mayor tolerancia a fallos puede impactar rendimiento	⚠️ Distribuir datos para tolerancia a fallos puede afectar consistencia	⚠️ Diseñar para fallos complica la mantenibilidad	✅ Un sistema confiable mejora la UX
Seguridad	⚠️ Medidas de seguridad pueden afectar disponibilidad	✅ Compatible	⚠️ Cifrado y auditoría impactan rendimiento	⚠️ Seguridad distribuida puede ralentizar escalabilidad	⚠️ Más controles de seguridad complican el mantenimiento	⚠️ Autenticación estricta puede afectar experiencia de usuario
Rendimiento	⚠️ Redundancia para confiabilidad puede afectar latencia	⚠️ Cifrado y logs pueden impactar la velocidad	✅ Compatible	✅ Distribuir carga mejora el rendimiento	⚠️ Código optimizado para rendimiento puede ser difícil de modificar	⚠️ Optimización extrema puede dificultar la interacción
Escalabilidad	⚠️ Alta disponibilidad complica escalabilidad	⚠️ Seguridad en sistemas distribuidos es más compleja	✅ Balanceo de carga mejora rendimiento	✅ Compatible	⚠️ Aumentar nodos puede dificultar mantenimiento	✅ UX estable en escalabilidad
Mantenibilidad	⚠️ Código preparado para fallos puede ser más difícil de modificar	⚠️ Medidas de seguridad complican actualización del sistema	⚠️ Código optimizado puede ser menos legible	⚠️ Sistemas altamente escalables suelen ser más complejos	✅ Compatible	✅ Una arquitectura mantenible permite cambios en la UI

Usabilidad	✅ Un sistema confiable mejora la UX	⚠️ Autenticación y validaciones pueden frustrar a los usuarios	⚠️ Alta optimización puede hacer la UI menos intuitiva	✅ Escalabilidad permite mantener buena UX con más usuarios	✅ Un sistema mantenible permite iteraciones en UX	✅ Compatible

✅ **Compatible** = Los atributos se complementan.
⚠️ **Conflicto** = Hay un trade-off entre ambos.

🔹 3. Ejemplos de Tensiones en Software Real

📌 Ejemplo 1: Plataforma Bancaria (Seguridad vs Usabilidad vs Rendimiento)

- **Se prioriza seguridad** con autenticación multifactor y encriptación de datos.
- **Impacto en usabilidad**: El usuario debe hacer más pasos para autenticarse.
- **Impacto en rendimiento**: Los procesos de cifrado aumentan el tiempo de respuesta.
- **Solución balanceada**: Implementar autenticación adaptativa (solo MFA en situaciones sospechosas).

📌 Ejemplo 2: Servicio de Streaming (Escalabilidad vs Consistencia vs Rendimiento)

- **Se prioriza escalabilidad** usando almacenamiento distribuido.
- **Impacto en consistencia**: Las películas guardadas en una lista pueden tardar en sincronizarse en diferentes dispositivos.
- **Impacto en rendimiento**: Se usan estrategias de caché para mejorar velocidad.
- **Solución balanceada**: Uso de **consistencia eventual** para evitar bloqueos.

📌 Ejemplo 3: Sistema de Respuesta de Emergencias (Fiabilidad vs Mantenibilidad vs Disponibilidad)

- **Se prioriza disponibilidad y confiabilidad**: El sistema debe funcionar 24/7.
- **Impacto en mantenibilidad**: No puede apagarse para actualizaciones.
- **Solución balanceada**: Implementar actualizaciones en caliente y redundancia de servidores.

🔹 4. ¿Cómo manejar estas tensiones?

1. **Definir prioridades del negocio** 🏛️
    - No todos los atributos son igual de importantes en cada sistema.
    - En una **banca en línea**, seguridad y fiabilidad son clave.
    - En un **juego en línea**, rendimiento y escalabilidad son críticos.

2. **Usar patrones arquitectónicos** 🏗️
    - **Patrón CQRS** ayuda a balancear escalabilidad y consistencia.
    - **Circuit Breaker** mejora disponibilidad sin comprometer rendimiento.
    - **Load Balancers** mejoran rendimiento sin degradar fiabilidad.
3. **Medir impacto con métricas** 📊
    - **Tiempo de respuesta** (rendimiento).
    - **Tiempo medio entre fallos (MTBF)** (fiabilidad).
    - **Tiempo medio de recuperación (MTTR)** (recuperabilidad).
    - **Latencia en transacciones seguras** (seguridad vs rendimiento).
4. **Aplicar estrategias de compromiso (Trade-offs)** ⚖️
    - **Consistencia eventual** en sistemas distribuidos.
    - **Autenticación adaptativa** para seguridad sin sacrificar usabilidad.
    - **Caché de datos** para mejorar rendimiento sin afectar escalabilidad.

🎯 Conclusión

✔ **No se pueden maximizar todos los atributos de calidad al mismo tiempo**. Siempre habrá **tensiones** entre algunos de ellos.

✔ **Las decisiones arquitectónicas deben considerar los trade-offs**. Elegir una solución óptima para un atributo puede comprometer otro.

✔ **Se deben aplicar patrones y estrategias** para mitigar conflictos. Una buena arquitectura busca el **mejor equilibrio según las necesidades del negocio**.